

MATH 448

**Exportable Ginger Percentage Based on Quality
Evaluation**

Carlos Suarez

2025

I. Abstract

This project, titled *Exportable Ginger Percentage Based on Quality Evaluation*, addresses a key operational challenge faced by a Peruvian ginger export company. The company sources ginger from independent farmers and determines payment based on the proportion of the harvest deemed exportable after processing (a process that takes about one week). This delay affects the farmers' cash flow and limits the company's ability to secure more ginger for export.

To solve this problem, I developed a predictive model using sample data collected at the factory's quality evaluation stage, which includes indicators such as sickness rate, weight per crate, and mechanical damage. The objective is to accurately forecast the exportable portion of each ginger batch upon arrival, allowing the company to compensate farmers promptly.

The analysis used real operational data from the entire 2024 calendar year. Six statistical learning methods were applied: Ordinary Least Squares, Best Subset Selection, Ridge Regression, Lasso Regression, Principal Components Regression (PCR), and Partial Least Squares (PLS). Among these, PLS and PCR achieved the best predictive performance, with PLS yielding the lowest test Mean Absolute Error (MAE) and Mean Squared Error (MSE).

The findings demonstrate that predictive modeling can significantly reduce the payment delay for farmers, enhancing supplier satisfaction and strengthening the company's competitive position in the ginger export market.

II. Introduction

In the global market for agricultural exports, speed and efficiency in the supply chain are crucial to maintaining competitiveness. This is particularly true in the Peruvian ginger industry, where exporters often depend on independent farmers to supply raw ginger. One such company, which exports ginger to the United States and Europe, currently faces a delay of approximately one week between receiving a batch and determining the amount of exportable ginger after processing. This delay directly impacts farmer satisfaction and supply stability, as payment is contingent upon the final processed yield.

Upon arrival at the company's processing facility, a quality control team samples the ginger and records several key indicators such as sickness level, mechanical damage, and weight per crate. However, without the ability to immediately estimate the exportable portion, the company cannot compensate farmers until processing is complete.

The goal of this project is to develop a reliable predictive model that estimates the exportable percentage of each ginger batch using only the initial quality evaluation data. By doing so, the company could reduce the payment time from several days to a matter of hours, thereby attracting more farmers, improving cash flow cycles, and increasing the volume of ginger available for export.

This analysis uses complete operational data from the year 2024 and applies a range of statistical learning methods to build and compare predictive models. The project aims not only to identify the best-performing model but also to provide actionable insights that support faster, data-driven decisions in the company's procurement process.

III. Data description, cleaning and preparation

This analysis is based on quality evaluation data collected throughout the 2024 calendar year from a Peruvian company that exports ginger. The dataset comprises approximately 1,000 records, each corresponding to a batch of ginger evaluated upon arrival at the company's processing facility. Data was sourced directly from the company's PostgreSQL database and reflects operational conditions, making it highly relevant for predictive modeling.

Variables

The dataset consists of one response variable and eight predictor variables, described as follows:

- **Response Variable:**
 - **Exportable Percentage:** The proportion of each ginger batch, by weight, that was deemed suitable for export after full processing.
- **Predictor Variables:**
 - **Mechanical Damage (%):** Portion of the sample affected by handling-related defects like cuts or breaks.
 - **Loss (%):** Material lost during the washing process, mostly comprising soil and unusable debris.
 - **Weight per Crate (kg):** Average weight of ginger per crate, an indicator of batch density and physical yield.
 - **Wart (%):** Share of the sample exhibiting wart-type deformities.
 - **Black Rot (%):** Proportion affected by black fungal rot.
 - **Rotten Tips (%):** Incidence of decay concentrated at the tips of ginger pieces.
 - **Insect Damage (%):** Extent of visible damage from pest infestation.
 - **Mold (%):** Surface mold presence on the sample, indicating spoilage.

Data Cleaning & Preparation

The initial dataset required a series of preprocessing steps to ensure analytical reliability:

- **Missing Values:** Records with missing values in any of the four variables were examined. In cases where the missingness was minimal and random, the affected records were removed. For variables with non-random missingness, further domain-specific investigation was conducted, though no significant imputation was ultimately required.
- **Data Partitioning:** The data was split into training and testing sets using a stratified approach to preserve the distribution of the response variable. This ensured fair evaluation of model performance on unseen data.

	exportable	weight_per_crate	shrinkage	wart	black_rot	rotten_tips	mechanical_damage	insect_damage	mold
1	0.7766811	14.82622	0.0486	0.027	0.00000000	0.039	0.0307	0.02000000	0.00000000
2	0.8659486	17.18784	0.0426	0.027	0.00000000	0.042	0.0349	0.01000000	0.00000000
3	0.7894737	16.71296	0.0354	0.030	0.01000000	0.040	0.0347	0.01000000	0.00000000
4	0.8129172	16.30294	0.0345	0.029	0.00000000	0.041	0.0310	0.00000000	0.00000000
5	0.7246377	16.20455	0.0336	0.032	0.01000000	0.044	0.0315	0.00000000	0.00000000
6	0.7380972	15.91410	0.0306	0.026	0.01000000	0.054	0.0320	0.00300000	0.00000000
7	0.6643009	16.01136	0.0339	0.036	0.00000000	0.054	0.0312	0.00000000	0.00000000
8	0.6377124	16.02094	0.0332	0.034	0.00000000	0.049	0.0305	0.02000000	0.00000000
9	0.8101218	16.42500	0.0348	0.026	0.00000000	0.045	0.0351	0.00000000	0.00000000
10	0.7539178	16.34938	0.0416	0.030	0.01000000	0.046	0.0308	0.00000000	0.00000000
11	0.7684794	17.76618	0.0368	0.025	0.02000000	0.044	0.0323	0.00000000	0.00000000
12	0.9056263	16.00145	0.0380	0.026	0.00000000	0.035	0.0363	0.01000000	0.00000000
13	0.8122305	15.94375	0.0300	0.030	0.02000000	0.041	0.0344	0.01000000	0.00000000
14	0.8191632	16.10672	0.0310	0.027	0.00000000	0.043	0.0358	0.01000000	0.00000000
15	0.8487458	15.52907	0.0349	0.023	0.00000000	0.045	0.0365	0.00000000	0.00000000

Figure 1: First 15 records of the ginger batch evaluations from the 2024 dataset, showing the response and predictor variables used for model training.

IV. Data Visualization

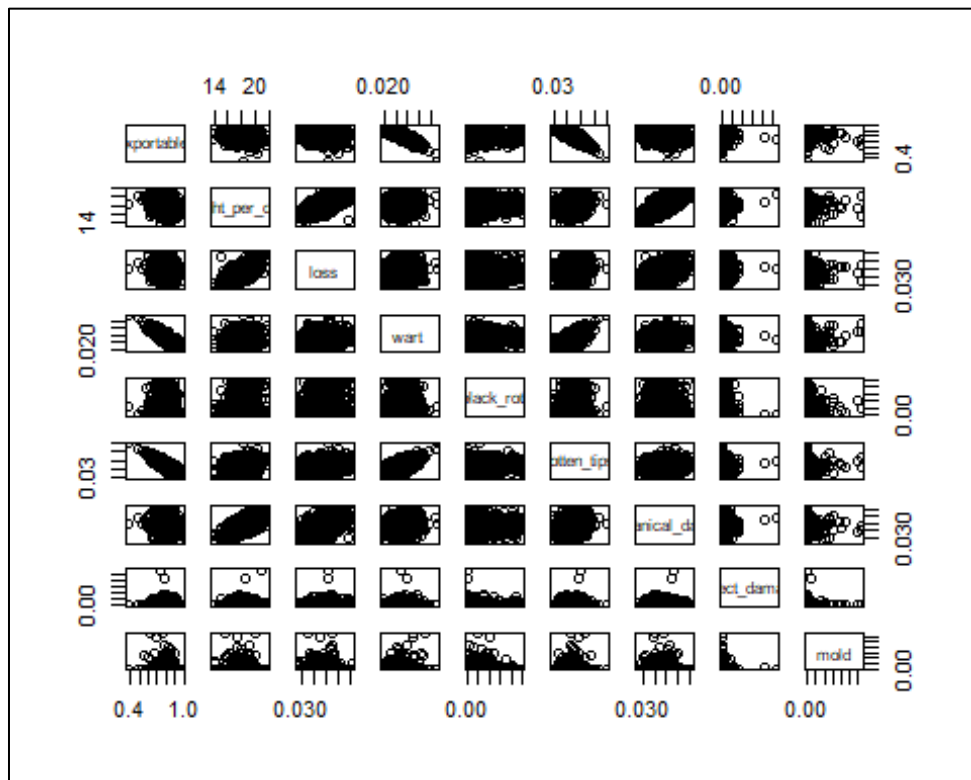


Figure 2: Scatterplot Matrix of All Variables

This matrix provides pairwise scatterplots for all variables, allowing visual inspection of potential linear relationships. Clear negative trends can be observed between *exportable*

percentage and predictors such as *wart*, *loss*, and *rotten tips*, suggesting these features are inversely related to the final exportable yield.

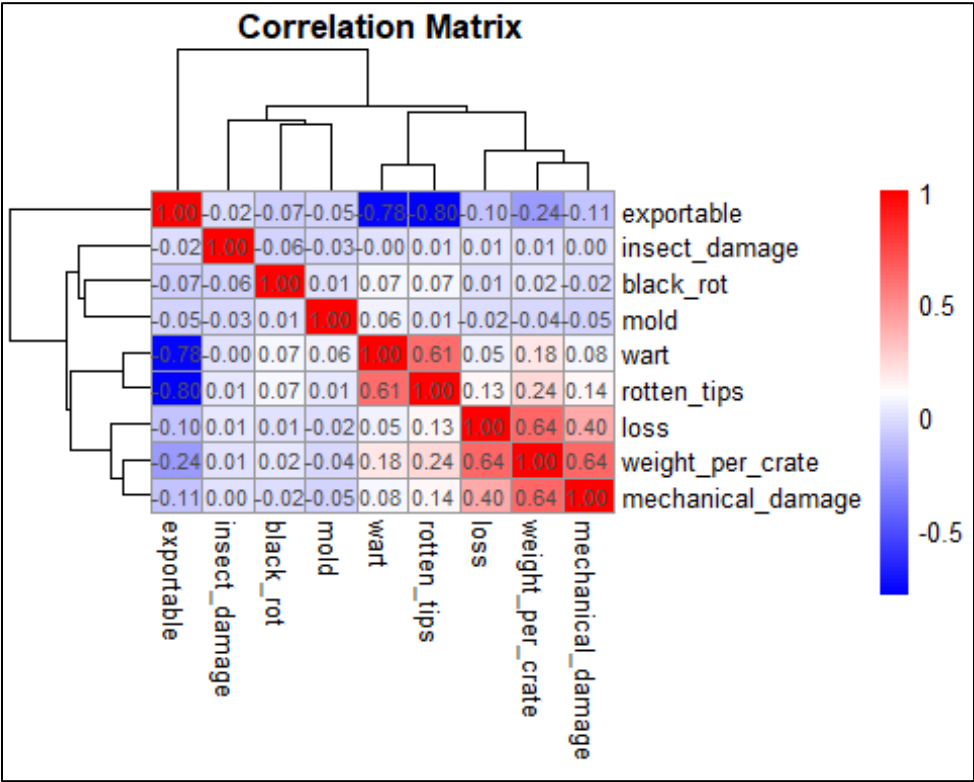


Figure 3: Correlation Matrix

The correlation heatmap shows the pairwise Pearson correlation coefficients between all variables, enhanced with dendrograms for variable clustering. The *exportable percentage* is strongly negatively correlated with *wart* ($r = -0.78$) and *shrinkage* ($r = -0.80$), supporting the insights from the scatterplot matrix.

V. METHODOLOGY

1. Ordinary Least Squares (OLS) Regression:

OLS regression was used as the baseline model. Initially, the model included all eight predictors. Based on the resulting p-values, a subset of statistically significant predictors was selected for a second model. These included: **weight per crate**, **wart**, **rotten tips**, and **mechanical damage**. The full model yielded an R-squared value of **0.7792**, while the reduced model yielded a slightly lower R-squared of **0.7781**. This minor drop is expected given the removal of four predictors, but the reduced model is more parsimonious and contains only predictors with statistically significant contributions to the response variable.

```

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)    1.377774   0.014523  94.867 < 2e-16 ***
weight_per_crate -0.004608  0.001187  -3.882 0.000109 ***
loss           0.434499   0.293600   1.480 0.139145
wart          -9.161215   0.333976 -27.431 < 2e-16 ***
black_rot       0.015297   0.039072   0.392 0.695490
rotten_tips    -6.451526   0.211241 -30.541 < 2e-16 ***
mechanical_damage 0.633153   0.301727   2.098 0.036063 *
insect_damage  -0.185692   0.155799  -1.192 0.233532
mold          -0.291712   0.190455  -1.532 0.125853
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.03233 on 1281 degrees of freedom
Multiple R-squared:  0.7792, Adjusted R-squared:  0.7778
F-statistic: 565 on 8 and 1281 DF, p-value: < 2.2e-16

```

Figure 4: Baseline model

```

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)    1.3775931   0.0145169  94.896 < 2e-16 ***
weight_per_crate -0.0036213  0.0009967  -3.633 0.000291 ***
wart          -9.2241497   0.3322166 -27.765 < 2e-16 ***
rotten_tips    -6.4395315   0.2111086 -30.503 < 2e-16 ***
mechanical_damage 0.6344619   0.3016064   2.104 0.035607 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.03235 on 1285 degrees of freedom
Multiple R-squared:  0.7781, Adjusted R-squared:  0.7775
F-statistic: 1127 on 4 and 1285 DF, p-value: < 2.2e-16

```

Figure 5: Model with the predictors that have a statistical effect on the response variable

2. Best Subset Selection:

It was used to evaluate all possible combinations of the eight predictor variables. The models were assessed using several statistical criteria: **Residual Sum of Squares (RSS)**, **Adjusted R-squared**, **Mallows' Cp**, and **Bayesian Information Criterion (BIC)**.

Upon analysis, the performance metrics showed only marginal differences between models with as few as two predictors and models with all eight. This suggested that including additional variables did not substantially improve model fit. Therefore, the model with the **lowest BIC**, which included **three predictors**, was selected. This more parsimonious model not only avoids overfitting but is also easier to interpret and communicate to non-technical stakeholders.

This decision underscores the importance of balancing model accuracy with simplicity, especially when the gains from complexity are minimal.

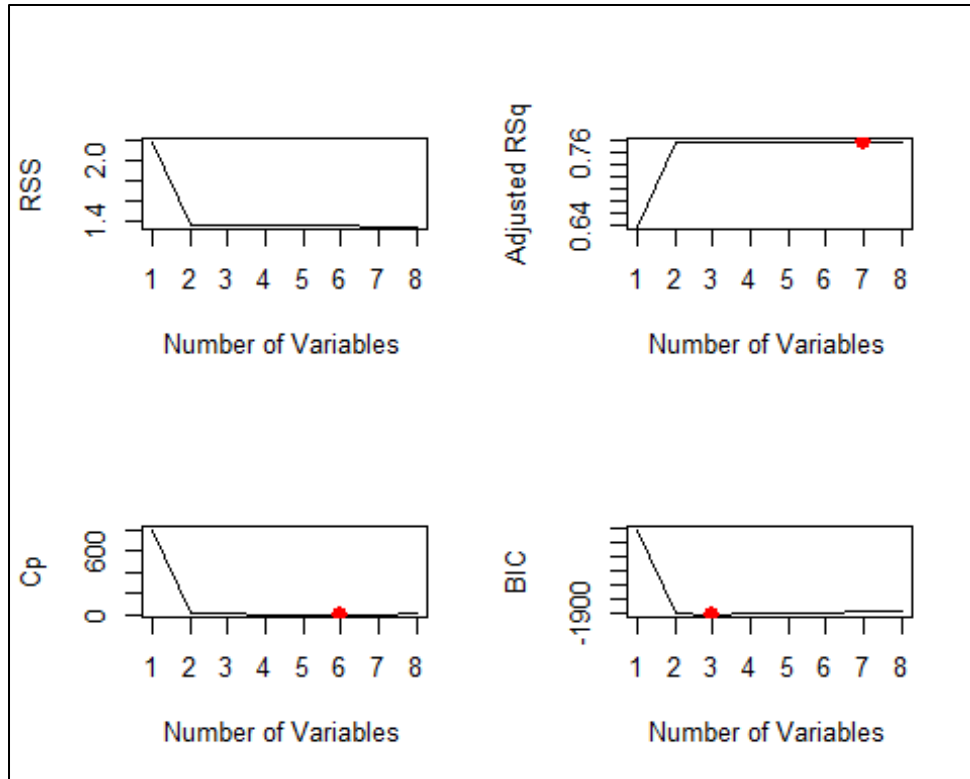


Figure 6: Plots of RSS, Adjusted R-squared, Mallow's Cp, and BIC across models with increasing numbers of predictors.

3. Principal Components Regression (PCR):

PCR was used to address multicollinearity among predictors by reducing them to a smaller set of uncorrelated components before fitting a linear regression model. These principal components are derived without considering the response variable, focusing solely on explaining the variance in the predictors.

To determine the optimal number of components to include, the **Mean Squared Error of Prediction (MSEP)** was evaluated across models using 1 to 8 components. As shown in the results, the lowest MSEP was achieved using **five components**, which were therefore selected for the final PCR model.

This approach provided a more stable and generalizable model, especially useful in settings where predictor variables are highly correlated, as is the case in this dataset.

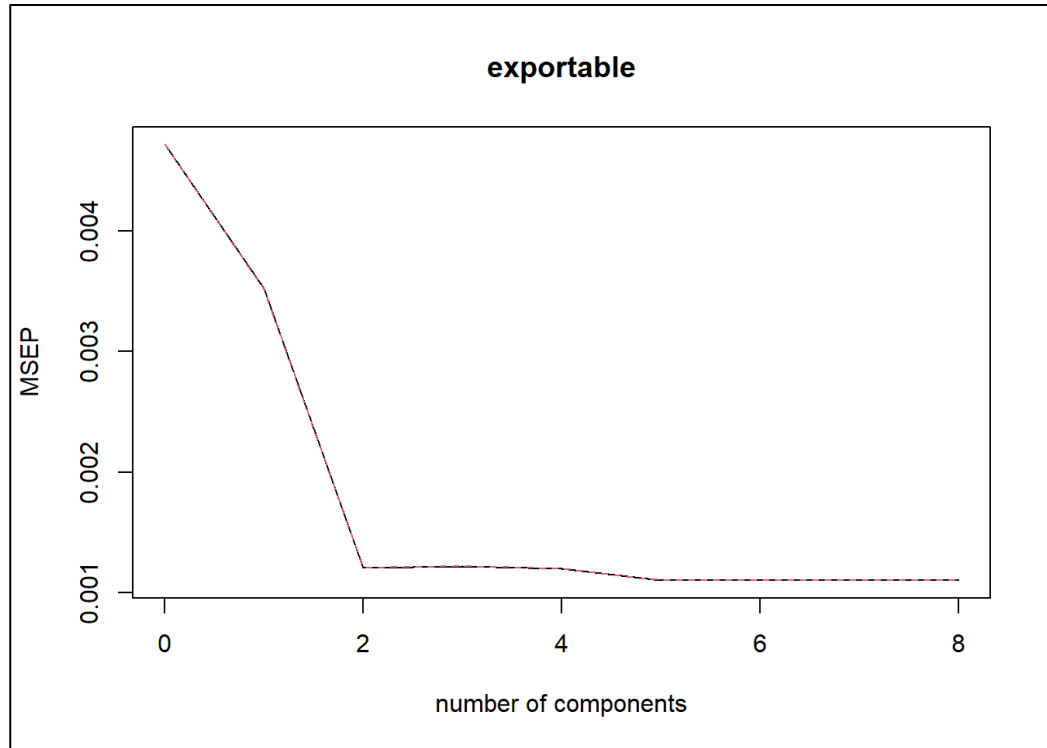


Figure 7: MSEP vs. Number of Principal Components

4. Partial Least Squares:

Partial Least Squares regression is a dimensionality reduction technique similar to PCR, but it differs in a critical way: it constructs components based on both the predictors and the response variable. This generally leads to better predictive performance when the primary goal is accurate prediction rather than just variance explanation.

To determine the optimal number of components, the **Mean Squared Error of Prediction (MSEP)** was calculated for models with 1 through 8 components. The lowest MSEP occurred with **2 components**, which were selected for the final PLS model.

This model ultimately achieved the best predictive performance across all evaluated methods, making it the most effective solution for estimating the exportable proportion of ginger based on intake sample characteristics.

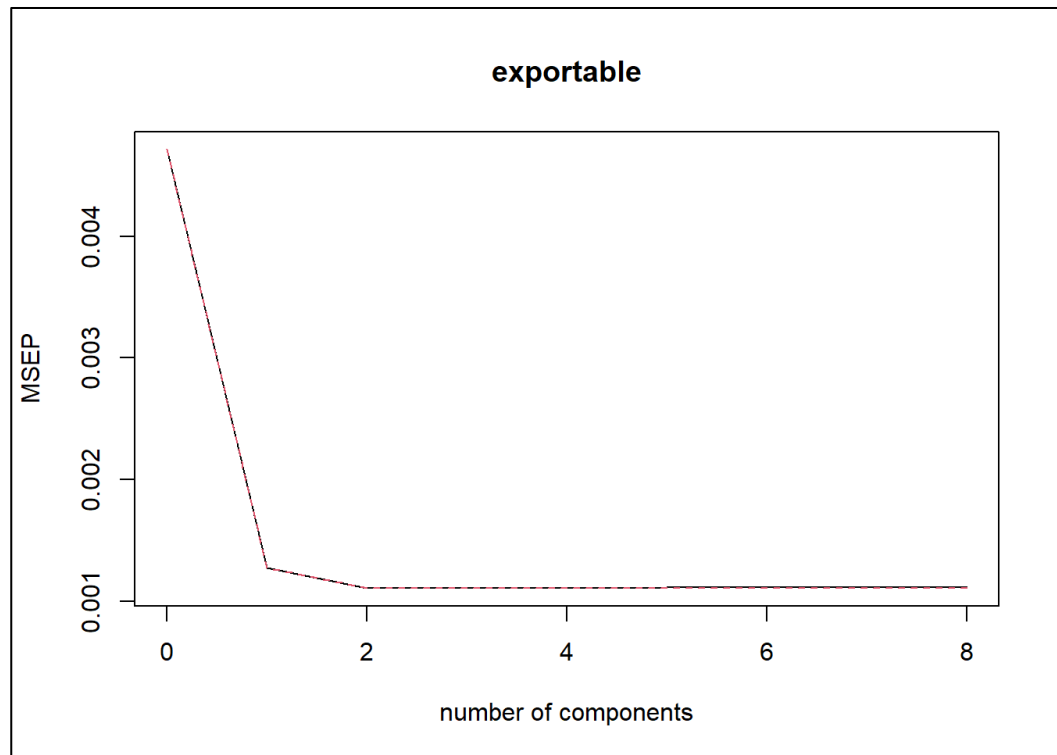


Figure 8: MSEP vs. Number of PLS Components

5. Ridge Regression:

Ridge regression was used to address multicollinearity among the predictor variables by introducing an L2 penalty term that shrinks the regression coefficients. This approach helps stabilize coefficient estimates and can reduce overfitting in models with correlated predictors.

To tune the regularization strength, the model was trained over a grid of lambda values using cross-validation. The **optimal lambda**, which minimized the **Mean Squared Error (MSE)**, was found to be **0.0055**. This value reflects the best trade-off between model complexity and predictive accuracy for the ridge regression model.

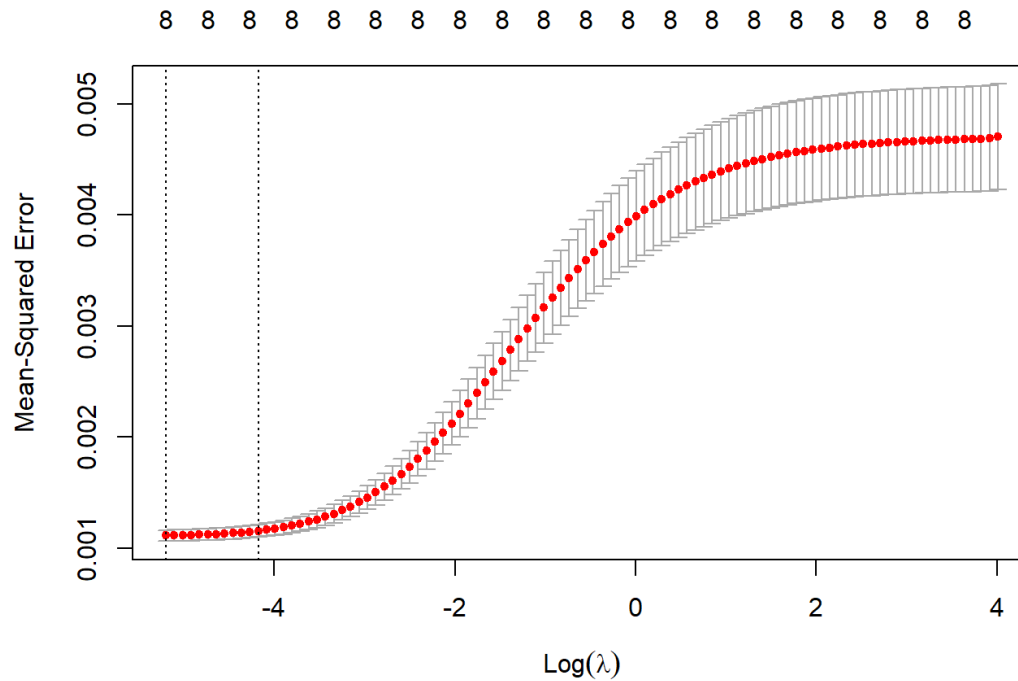


Figure 9: Mean-Squared Error vs. Log(Lambda) for Ridge Regression

6. Lasso Regression

Lasso regression was used to perform both coefficient shrinkage and variable selection by applying an L1 penalty to the regression model. Unlike ridge regression, lasso can reduce some coefficients exactly to zero, effectively removing less relevant predictors and producing a more interpretable model.

The model was trained across a range of lambda values using cross-validation. The lambda that resulted in the **lowest Mean Squared Error (MSE)** was **0.0001084**. At this value, the lasso model achieved a strong balance between prediction accuracy and sparsity, identifying a subset of influential predictors while eliminating others.

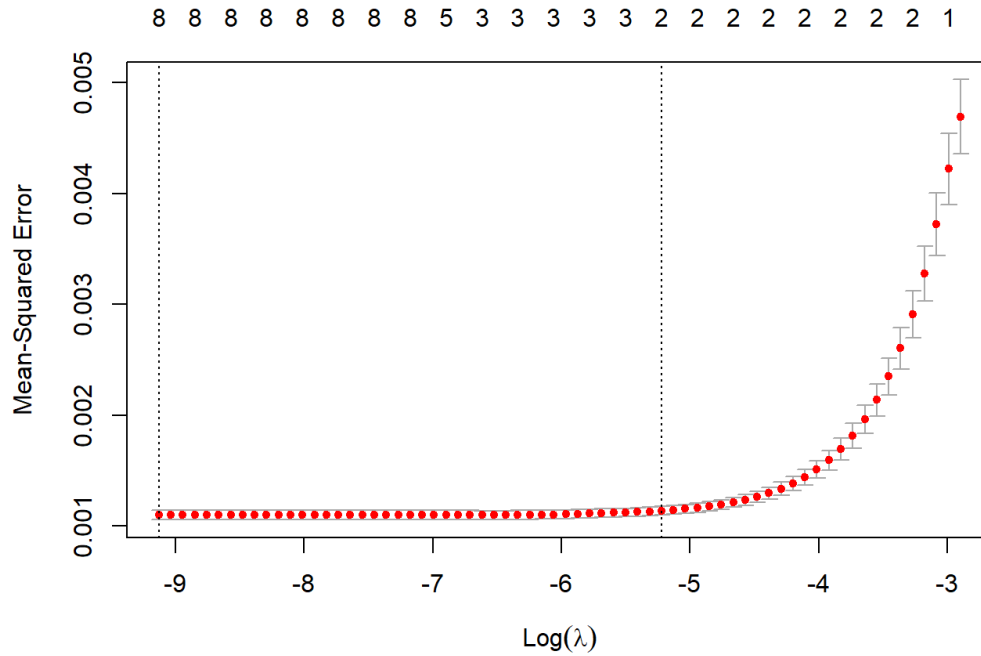


Figure 10: Mean-Squared Error vs. Log(Lambda) for Lasso Regression

VI. RESULTS

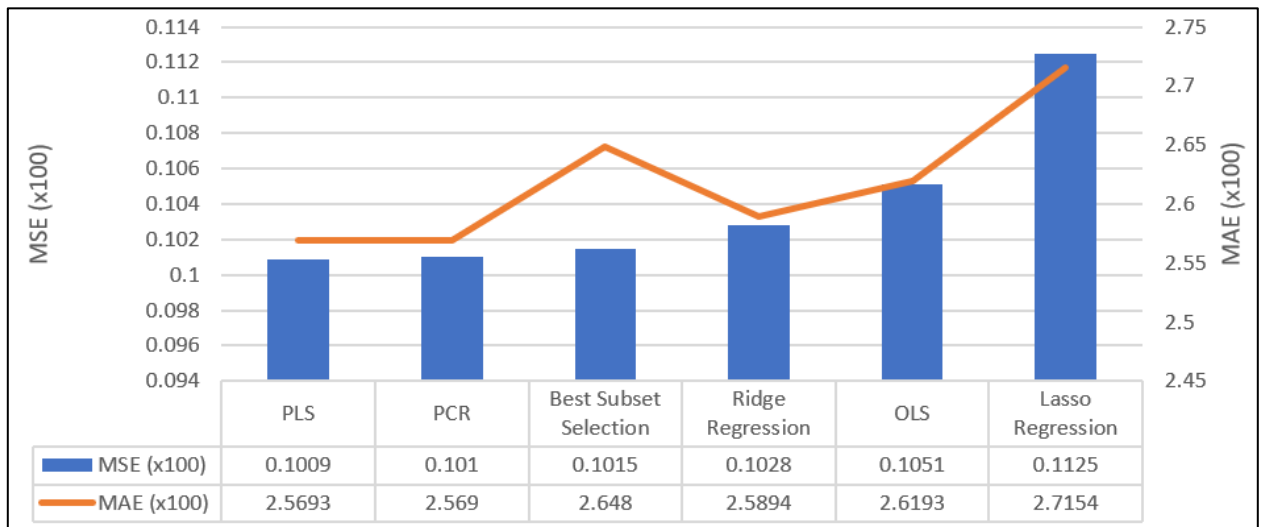


Figure 11: Test Set Performance of Predictive Models

To evaluate the performance of the six predictive models, two standard metrics were used on the test dataset: **Mean Squared Error (MSE)** and **Mean Absolute Error (MAE)**. These metrics quantify the average magnitude of prediction errors, with lower values indicating better predictive performance.

Figure 11 displays the test MSE and MAE for each model. The results confirm that **Partial Least Squares (PLS)** achieved the best overall performance, with the lowest MSE and MAE among all methods. **Principal Components Regression (PCR)** followed closely, showing nearly identical error rates. These two models demonstrate that dimensionality reduction approaches are particularly effective for this dataset, where predictor variables are often correlated.

In contrast, **Lasso Regression** produced the highest test error values, suggesting that its variable elimination strategy may have discarded relevant information. Traditional linear models, including **OLS**, **Ridge Regression**, and **Best Subset Selection**, yielded intermediate performance, with Best Subset performing slightly better due to its simplicity and focus on the most informative variables.

VII. CONCLUSION AND FUTURE WORK

This project addressed a key operational inefficiency in the Peruvian ginger export industry: the delay between ginger delivery and payment to farmers, which currently depends on post-processing exportability assessment. By developing predictive models based on quality evaluation data collected at intake, this analysis enables the estimation of exportable yield much earlier in the process, potentially on the same day the ginger arrives at the factory.

Among the six statistical learning methods evaluated, **Partial Least Squares (PLS)** emerged as the most effective, achieving the lowest test Mean Squared Error (MSE) and Mean Absolute Error (MAE). **Principal Components Regression (PCR)** followed closely. These results underscore the importance of using dimensionality reduction techniques in datasets with multicollinearity. Simpler models like **Best Subset Selection** also performed well, offering interpretability with minimal loss in predictive power.

While the results are promising, several opportunities exist to extend and improve this analysis:

- **Temporal Extension:** The ginger crop cycle spans nearly a full year. As more data from the current year's harvest becomes available, incorporating it could uncover temporal trends and seasonal patterns that affect exportable yield.
- **Field-Level Sampling:** Since four months ago, employees have begun collecting samples during the harvesting process. Integrating this **pre-intake field data** could provide earlier quality indicators, allowing even faster and more informed decisions.
- **New Predictors:** Additional variables such as **ginger ripeness** and **harvest zone** may influence exportability. Including such agronomic and geographic information could improve model accuracy and offer deeper insights into yield variability.
- **Classification Models:** Beyond predicting exportable percentage, a binary classification model could be developed to determine whether a batch should be **accepted or rejected** for processing. This would add operational value by supporting real-time quality control and supply chain decision-making.

Together, these enhancements point to a robust data-driven strategy for modernizing procurement and quality assurance in ginger exports, delivering both **competitive advantage** and **improved farmer relationships**.

VIII.CODE APPENDIX/SUPPLEMENT

#1. Read and prepare data

```
data0 <- read_excel("C:/Users/carl0/Documents/SFSU/2nd Semester/Math 448 - Introduction to Statistical Learning and Data Mining/Project/DB Project.xlsx",  
sheet = "DB")  
data <- data0[, 3:11] # Select relevant columns  
n <- nrow(data)
```

```
head(data)
```

```
## # A tibble: 6 × 9  
##   exportable weight_per_crate   loss   wart black_rot rotten_tips  
##   <dbl>         <dbl> <dbl> <dbl>    <dbl>    <dbl>  
## 1     0.777         14.8 0.0486 0.027      0     0.039  
## 2     0.866         17.2 0.0426 0.027      0     0.042  
## 3     0.789         16.7 0.0354 0.03      0.01    0.04  
## 4     0.813         16.3 0.0345 0.029      0     0.041  
## 5     0.725         16.2 0.0336 0.032      0.01    0.044  
## 6     0.738         15.9 0.0306 0.026      0.01    0.054  
## # i 3 more variables: mechanical_damage <dbl>, insect_damage <dbl>, mold <dbl>
```

#2. Plots

```
pairs(data)
```

```
cor_matrix<-cor(data)
```

```
pheatmap(cor_matrix,  
  display_numbers = TRUE, # Show the correlation coefficients  
  color = colorRampPalette(c("blue", "white", "red"))(100), # Color scale from blue to red  
  main = "Correlation Matrix")
```

#3. Methods:

```
# -----  
# Method 1: Linear Regression  
# -----  
# Fit full and reduced models  
data.lm <- lm(exportable ~ weight_per_crate + loss + wart + black_rot + rotten_tips + mechanical_damage + insect_damage + mold, data = data)  
summary(data.lm)  
  
##  
## Call:
```

```
## lm(formula = exportable ~ weight_per_crate + loss + wart + black_rot +
##     rotten_tips + mechanical_damage + insect_damage + mold, data = data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.134000 -0.022266  0.000384  0.021788  0.102353
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    1.377774    0.014523   94.867 < 2e-16 ***
## weight_per_crate -0.004608    0.001187   -3.882 0.000109 ***
## loss           0.434499    0.293600    1.480 0.139145
## wart          -9.161215    0.333976  -27.431 < 2e-16 ***
## black_rot       0.015297    0.039072    0.392 0.695490
## rotten_tips     -6.451526    0.211241  -30.541 < 2e-16 ***
## mechanical_damage 0.633153    0.301727    2.098 0.036063 *
## insect_damage   -0.185692    0.155799   -1.192 0.233532
## mold           -0.291712    0.190455   -1.532 0.125853
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.03233 on 1281 degrees of freedom
## Multiple R-squared:  0.7792, Adjusted R-squared:  0.7778
## F-statistic: 565 on 8 and 1281 DF, p-value: < 2.2e-16

data.lm2 <- lm(exportable ~ weight_per_crate + wart + rotten_tips + mechanical_
L_damage, data = data)
summary(data.lm2)

##
## Call:
## lm(formula = exportable ~ weight_per_crate + wart + rotten_tips +
##     mechanical_damage, data = data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.131859 -0.022366  0.000265  0.021917  0.104804
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    1.3775931    0.0145169   94.896 < 2e-16 ***
## weight_per_crate -0.0036213    0.0009967   -3.633 0.000291 ***
## wart          -9.2241497    0.3322166  -27.765 < 2e-16 ***
## rotten_tips     -6.4395315    0.2111086  -30.503 < 2e-16 ***
## mechanical_damage 0.6344619    0.3016064    2.104 0.035607 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.03235 on 1285 degrees of freedom
```

```
## Multiple R-squared:  0.7781, Adjusted R-squared:  0.7775
## F-statistic: 1127 on 4 and 1285 DF,  p-value: < 2.2e-16
```

```
# Evaluate performance on a test set
lm.test <- data[1191:1290, ]
lm.pred <- predict(data.lm2, lm.test)
ss_res <- sum((lm.test$exportable - lm.pred)^2)
ss_tot <- sum((lm.test$exportable - mean(lm.test$exportable))^2)
r_squared <- 1 - ss_res / ss_tot
mse_lm <- mean((lm.test$exportable - lm.pred)^2)
mae_lm <- mean(abs(lm.test$exportable - lm.pred))
r_squared

## [1] 0.6429799

mse_lm

## [1] 0.001006032

mae_lm

## [1] 0.02640318

formula_ols <- exportable ~ weight_per_crate + wart + rotten_tips + mechanical_damage

# Define custom CV function for both MSE and MAE
cv_ols_mae_mse <- function(data, formula, K = 10) {
  n <- nrow(data)
  folds <- sample(rep(1:K, length = n))
  mse <- rep(NA, K)
  mae <- rep(NA, K)

  for (k in 1:K) {
    train_idx <- which(folds != k)
    test_idx <- which(folds == k)

    model <- lm(formula, data = data[train_idx, ])
    preds <- predict(model, newdata = data[test_idx, ])
    actuals <- data$exportable[test_idx]

    mse[k] <- mean((preds - actuals)^2)
    mae[k] <- mean(abs(preds - actuals))
  }

  return(list(mse = mean(mse), mae = mean(mae)))
}

# Apply the custom CV function
cv_results <- cv_ols_mae_mse(data, formula_ols, K = 10)
```

```

# Output
cat("10-fold CV MSE:", cv_results$mse, "\n")

## 10-fold CV MSE: 0.001051269

cat("10-fold CV MAE:", cv_results$mae, "\n")

## 10-fold CV MAE: 0.02621079

# LOOCV and 10-fold CV
glm.model <- glm(exportable ~ weight_per_crate + wart + rotten_tips + mechani
cal_damage, data = data)
cv_loocv <- cv.glm(data, glm.model)
cv_kfold <- cv.glm(data, glm.model, K = 10)
cv_loocv$delta[1]

## [1] 0.001050881

cv_kfold$delta[1]

## [1] 0.001050567

# -----
# Method 2: Subset Selection
# -----
regfit.full <- regsubsets(exportable ~ ., data = data, nvmax = 8)
reg.summary <- summary(regfit.full)

# Cross-validation for best subset selection
predict.regsubsets <- function(object, newdata, id, ...) {
  form <- as.formula(object$call[[2]])
  mat <- model.matrix(form, newdata)
  coefi <- coef(object, id = id)
  xvars <- names(coefi)
  mat[, xvars] %*% coefi
}

k <- 10
set.seed(1)
folds <- sample(1:k, n, replace = TRUE)
cv.errors <- matrix(NA, k, 8)

for (j in 1:k) {
  best.fit <- regsubsets(exportable ~ ., data = data[folds != j, ], nvmax = 8
)
  for (i in 1:8) {
    pred <- predict.regsubsets(best.fit, data[folds == j, ], id = i)
    cv.errors[j, i] <- mean((data$exportable[folds == j] - pred)^2)
  }
}

```

```

mean.cv.errors <- apply(cv.errors, 2, mean)
best.model.size <- which.min(mean.cv.errors)
min.cv.mse.subset <- mean.cv.errors[best.model.size]
min.cv.mse.subset

## [1] 0.001051637

# Evaluate subset selection on test set
reg.best <- regsubsets(exportable ~ ., data = data, nvmax = 8)
best.coefs <- coef(reg.best, best.model.size)
xvars <- names(best.coefs)
X_test <- model.matrix(as.formula(paste("exportable ~", paste(xvars[-1], collapse = "+"))), data = lm.test)
y_pred_subset <- X_test %*% best.coefs
mse_subset <- mean((lm.test$exportable - y_pred_subset)^2)
mae_subset <- mean(abs(lm.test$exportable - y_pred_subset))
mse_subset

## [1] 0.001014948

mae_subset

## [1] 0.0264797

par(mfrow=c(2,2))
plot(reg.summary$rss,xlab="Number of Variables",ylab="RSS",type="l")
which.min(reg.summary$rss)

## [1] 8

points(8,reg.summary$adjr2[8], col="red",cex=2,pch=20)
plot(reg.summary$adjr2,xlab="Number of Variables",ylab="Adjusted RSq",type="l")
which.max(reg.summary$adjr2)

## [1] 7

points(7,reg.summary$adjr2[7], col="red",cex=2,pch=20)
plot(reg.summary$cp,xlab="Number of Variables",ylab="Cp",type='l')
which.min(reg.summary$cp)

## [1] 6

points(6,reg.summary$cp[6],col="red",cex=2,pch=20)
which.min(reg.summary$bic)

## [1] 3

plot(reg.summary$bic,xlab="Number of Variables",ylab="BIC",type='l')
points(3,reg.summary$bic[3],col="red",cex=2,pch=20)

```

```

coef(reg.best, 3)

##      (Intercept) weight_per_crate      wart      rotten_tips
##      1.379038615      -0.002303865     -9.254326170     -6.435322179

# -----
# Method 3 and 4: PCR and PLS
# -----
x <- model.matrix(exportable ~ ., data)[, -1]
y <- data$exportable
set.seed(1)
train <- sample(1:nrow(x), nrow(x)/2)
test <- (-train)
y.test <- y[test]

# PCR
pcr.fit <- pcr(exportable ~ ., data = data, subset = train, scale = TRUE, validation = "CV")
summary(pcr.fit)

## Data:      X dimension: 645 8
## Y dimension: 645 1
## Fit method: svdpc
## Number of components considered: 8
##
## VALIDATION: RMSEP
## Cross-validated using 10 random segments.
##      (Intercept)  1 comps  2 comps  3 comps  4 comps  5 comps  6 comps
## CV      0.06867  0.05932  0.03476  0.03496  0.03466  0.03326  0.03327
## adjCV    0.06867  0.05929  0.03471  0.03493  0.03463  0.03323  0.03325
##      7 comps  8 comps
## CV      0.03323  0.03324
## adjCV    0.03320  0.03321
##
## TRAINING: % variance explained
##      1 comps  2 comps  3 comps  4 comps  5 comps  6 comps  7 comps
## X      28.58   47.45   60.83   73.28   84.44   92.18   96.80
## exportable 25.68   74.32   74.46   74.96   77.06   77.06   77.19
##      8 comps
## X      100.00
## exportable 77.26

par(mfrow = c(1, 1))
validationplot(pcr.fit, val.type = "MSEP")

pcr.pred <- predict(pcr.fit, x[test, ], ncomp = 5)
mse_pcr <- mean((pcr.pred - y.test)^2)
mae_pcr <- mean(abs(pcr.pred - y.test))
mse_pcr

```

```
## [1] 0.001010317

mae_pcr

## [1] 0.02569031

# PLS
pls.fit <- plsrf(exportable ~ ., data = data, subset = train, scale = TRUE, validation = "CV")
summary(pls.fit)

## Data:      X dimension: 645 8
## Y dimension: 645 1
## Fit method: kernelppls
## Number of components considered: 8
##
## VALIDATION: RMSEP
## Cross-validated using 10 random segments.
##      (Intercept)  1 comps  2 comps  3 comps  4 comps  5 comps  6 comps
## CV           0.06867  0.03567  0.03328  0.03327  0.03330  0.03332  0.03332
## adjCV        0.06867  0.03564  0.03326  0.03324  0.03327  0.03329  0.03328
##      7 comps  8 comps
## CV           0.03332  0.03332
## adjCV        0.03328  0.03328
##
## TRAINING: % variance explained
##      1 comps  2 comps  3 comps  4 comps  5 comps  6 comps  7 comps
## X           24.07   46.83   58.34   63.76   74.91   79.83   92.00
## exportable   73.37   77.03   77.22   77.26   77.26   77.26   77.26
##      8 comps
## X           100.00
## exportable   77.26

validationplot(pls.fit, val.type = "MSEP")

pls.pred <- predict(pls.fit, x[test, ], ncomp = 2)
mse_pls <- mean((pls.pred - y.test)^2)
mae_pls <- mean(abs(pls.pred - y.test))
mse_pls

## [1] 0.001009201

mae_pls

## [1] 0.0256934

# -----
# Method 5 and 6: Ridge and Lasso Regression
# -----
```

#Ridge

```
library(glmnet)
x <- model.matrix(exportable~., data)
y <- data$exportable
set.seed(1)
train <- sample(1:n, n/2)
test <- (-train)
y.test <- y[test]

grid <- 10^seq(10, -2, length = 100)
ridge.mod <- glmnet(x[train, ], y[train], alpha = 0,
  lambda = grid, thresh = 1e-12)

cv.out <- cv.glmnet(x[train, ], y[train], alpha = 0)
plot(cv.out)
```

```
bestlam <- cv.out$lambda.min
bestlam

## [1] 0.005523601

ridge.pred <- predict(ridge.mod, s = bestlam,
  newx = x[test, ])
mse_ridge <- mean((ridge.pred - y.test)^2)
mae_ridge <- mean(abs(ridge.pred - y.test))
mse_ridge

## [1] 0.0010278

mae_ridge

## [1] 0.0258936
```

#Lasso

```
lasso.mod <- glmnet(x[train, ], y[train], alpha = 1, lambda = grid)
set.seed(1)
cv.out <- cv.glmnet(x[train, ], y[train], alpha = 1)
plot(cv.out)

bestlam <- cv.out$lambda.min
bestlam

## [1] 0.0001084305

lasso.pred <- predict(lasso.mod, s = bestlam,
  newx = x[test, ])
mse_lasso <- mean((lasso.pred - y.test)^2)
```

```
mae_lasso <- mean(abs(lasso.pred - y.test))
mse_lasso

## [1] 0.001125403

mae_lasso

## [1] 0.02715356
```